

CSCE 614 COMPUTER ARCHITECTURE · MODULE 09 · LECTURE NOTES

SIMD and Data-Level Parallelism

One instruction, many elements.

MODULE OUTCOMES

- Explain SIMD execution and its register widths.
- Identify loops that can and cannot vectorise.
- Diagnose why a compiler failed to auto-vectorise.
- Use intrinsics when auto-vectorisation is insufficient.
- Explain why SoA layout is a precondition for good vectorisation.

1 Single instruction, multiple data

A scalar add instruction performs one addition and costs a fetch, a decode, and a slot in the execution pipeline. A vector add performs eight additions for the same overhead. The arithmetic units are wider, but the control overhead is unchanged — which is why SIMD is such an efficient way to add throughput.

Extension	Register width	Floats per op	Availability
SSE/SSE2	128-bit	4	Every x86-64 processor. Baseline.
AVX / AVX2	256-bit	8	Essentially universal since ~2013. The practical default target.
AVX-512	512-bit	16	Server and some desktop parts. Historically triggered clock reduction, which could make it a net loss for short vectorised sections.
NEON	128-bit	4	Every ARM64 processor. Baseline.
SVE / SVE2	Scalable (128–2048)	Varies	ARM's width-agnostic design: the same binary adapts to the hardware's actual width, avoiding recompilation per target.

2 What prevents vectorisation

Compilers vectorise automatically and do it well. When they fail, there is almost always a specific, identifiable reason, and the compiler will usually tell you what it is.

Blocker	Why it blocks	Remedy
Pointer aliasing	The compiler cannot prove the output array does not overlap an input. If they overlapped, processing eight elements at once would produce a different result than processing them one at a time.	restrict (C) or __restrict (C++). This is by far the most common blocker and the cheapest fix.
Loop-carried dependence	Iteration i needs a value produced by iteration i-1.	Often unavoidable. Sometimes the loop can be restructured, or the dependence confined to a scalar prologue.
Floating-point reduction	Vectorising requires reassociating additions, which changes the result.	#pragma omp simd reduction(+:sum) for one loop, or -ffast-math globally. See §2.2.
Branches in the loop body	Different lanes would need different control flow.	Convert to branchless arithmetic or masked operations, which modern compilers often do automatically.
Non-unit stride	Elements are not contiguous, so a vector load would require a gather — slow where supported at all.	Change the data layout (§3). This is usually the real fix.
Opaque function calls	The compiler cannot vectorise across a call it cannot see into.	Make the definition visible so it can be inlined.

2.1 Aliasing

```
// Does NOT vectorise: c might overlap a or b
void add(float* c, float* a, float* b, int n) {
    for (int i = 0; i < n; i++) c[i] = a[i] + b[i];
}

// Vectorises: restrict promises no overlap
void add(float* restrict c, float* restrict a,
         float* restrict b, int n) {
    for (int i = 0; i < n; i++) c[i] = a[i] + b[i];
}
```

RESTRICT IS AN UNCHECKED PROMISE

The compiler does not verify it; it simply trusts you and optimises accordingly. If the pointers do overlap, the behaviour is undefined and the symptom is typically silent data corruption that depends on optimisation level.

Use it where you genuinely know the arrays are distinct, which in practice is most numerical kernels. Do not scatter it hopefully.

2.2 Floating-point reductions

Summing an array into a single accumulator is a loop-carried dependence. Vectorising it means keeping eight partial sums and combining them at the end, which reassociates the additions.

$$(a + b) + c \neq a + (b + c) \quad \text{in floating point}$$

The results differ, usually in the last bits. A compiler will not make that change without permission, because silently altering numerical output is not a decision it is entitled to make.

Worth knowing: the vectorised sum is usually *more* accurate than the sequential one, because eight partial sums each stay smaller and therefore lose less precision when adding small values to a large running total. The compiler's caution is about *reproducibility*, not accuracy.

Prefer the narrow grant — `#pragma omp simd reduction(+:sum)` on the specific loop — over `-ffast-math`, which also enables assuming no NaNs and no infinities, among other things that can break code in surprising ways.

3 Layout determines vectorisability

```
// Array of structs: x values 32 bytes apart
struct P { float x,y,z, vx,vy,vz; int id; float m; };
P p[N];
for (i...) p[i].x += p[i].vx * dt;

// Struct of arrays: x values contiguous
struct Ps { float x[N], y[N], z[N], vx[N], vy[N], vz[N]; };
for (i...) P.x[i] += P.vx[i] * dt;
```

In the AoS version, the eight x values a vector register needs are scattered 32 bytes apart. Loading them requires either a gather instruction — which is available on recent hardware but far slower than a contiguous load — or eight separate loads followed by shuffle instructions to assemble the register. Either way most of the benefit evaporates.

In the SoA version a single aligned 256-bit load fetches all eight values. The loop vectorises cleanly and runs at close to the theoretical speedup.

TWO INDEPENDENT ARGUMENTS, ONE CONCLUSION

Module 06 argued for struct-of-arrays on cache grounds: you fetch only the bytes you use.

This module argues for it on vectorisation grounds: contiguous values load into vector registers in one instruction.

These are genuinely independent arguments, and they compound. A loop converted from AoS to SoA frequently gains more than either effect alone would predict, which is a good reason to treat layout as the first thing to examine rather than the last.

4 A working method

- 1 **Check whether it already vectorised.** `-fopt-info-vec` (GCC) or `-Rpass=loop-vectorize` (Clang) will report. Or read the assembly on Compiler Explorer and look for ymm registers.
- 2 **If it did not, find out why.** `-fopt-info-vec-missed` usually names the reason directly — compilers are better at explaining this than most people expect.
- 3 **Remove the blocker.** Add `restrict`, make functions inlinable, change the layout, or grant reduction permission.
- 4 **Only then consider intrinsics.** They are unportable, verbose, and must be maintained against each instruction-set target. Modern auto-vectorisation is good; reach past it only when you have measured that it is not good enough here.
- 5 **Measure.** Always.

WHY STEP 5 MATTERS MORE THAN THE OTHERS

SIMD multiplies *arithmetic* throughput. If the loop is waiting on memory, there was no arithmetic shortage to relieve.

Compute the **arithmetic intensity**: floating-point operations per byte loaded from memory. Below roughly 1 flop/byte, the loop is almost certainly bandwidth-bound, and vectorising the arithmetic will produce no measurable change — a result that surprises people who have just spent a day writing intrinsics.

This is precisely why Module 06 precedes Module 09. Fix memory behaviour first; vectorise what remains.

The partial exception: vector loads also move more bytes per instruction, so there can be a modest gain even when bandwidth-bound. Modest meaning perhaps 10–20%, not 8×.

5 Connections

Elsewhere	Relationship
CSCE 641	Fragment and vertex shaders are SIMD programs. A GPU warp is 32-wide SIMD with a programming model that hides the width — which is why divergent branches in a shader cost what they do.
Module 10	GPUs take data-level parallelism much further, with thousands of concurrent threads in addition to width.
CSCE 735	SIMD is the finest grain in the parallelism hierarchy: vector lanes, then cores, then sockets, then nodes. Each level has its own cost model.
Module 06	SoA layout is required for both cache efficiency and vectorisability. Do it once, benefit twice.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Agner Fog — Optimizing subroutines in assembly language	The reference for vector instruction selection, latencies, and throughputs.
Intel Intrinsics Guide	Searchable reference for every intrinsic, with latency and throughput per microarchitecture.
Onur Mutlu — SIMD Processing lectures	Vector processors and SIMD from the architectural side, including the historical vector supercomputers.
Mike Acton — Data-Oriented Design and C++ (free video)	Why layout decisions dominate, argued from shipped game engine experience.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a simple elementwise loop without `restrict` and confirm with compiler reports that it does not vectorise. Add `restrict` and measure the speedup.
- 2 Write a sum reduction and confirm it does not vectorise by default. Enable it with a `pragma`, then compare the numerical results against the scalar version and against a high-precision reference. Which is more accurate?
- 3 Convert a particle update from AoS to SoA. Measure the speedup and determine how much comes from cache behaviour and how much from vectorisation, by testing an SoA version with vectorisation disabled.
- 4 Compute the arithmetic intensity of three loops of your own. Predict which will benefit from vectorisation, then measure and check.
- 5 Write a kernel with intrinsics and compare it against the auto-vectorised version. Report whether the extra effort paid.
- 6 Write a loop with a data-dependent branch inside and examine whether the compiler vectorises it with masking. Compare against a branchless rewrite.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does SIMD amortise, and why does that make it efficient?
- 2 Name four things that block auto-vectorisation.
- 3 What does `restrict` promise, and what happens if you lie?
- 4 Why will a compiler not vectorise a floating-point reduction by default, and is the vectorised result less accurate?
- 5 Give both reasons SoA beats AoS, and say why they are independent.
- 6 You vectorise a loop and nothing gets faster. What is the most likely explanation and how would you confirm it?